

Maximal Semantic Clique Enumeration in Vector-Attributed Graphs

Tianyi Gao
Beijing Institute of Technology
Beijing, China
tygao_bit@163.com

Hongchao Qin
Beijing Institute of Technology
Beijing, China
qhc_neu@gmail.com

ABSTRACT

Maximal clique enumeration is a fundamental primitive for graph analysis, yet structural completeness alone cannot identify semantically coherent groups in vector-attributed graphs. Adding a collective similarity constraint makes the problem substantially harder: average pairwise similarity is non-monotone over the subset lattice, so the pivot invariant behind scalable structural enumeration no longer holds. We formalize maximal semantic clique (MSC) enumeration under inclusion-wise maximality and develop two exact algorithms. STRSUB restores structural pivoting by separating structural MCE from bounded semantic subset enumeration. MONOSEMCE targets selective thresholds through a canonical-parent theorem: deleting a minimum-contribution vertex from any feasible clique yields a unique feasible parent. The resulting reverse search generates each feasible state once without visited tables or materialized size layers. Across eight datasets and a fixed similarity-quantile grid, the two methods complete all 24 workloads, while the exact SEMBK baseline reaches the 1,200-second limit on 15. At q_{99} , MONOSEMCE is $2.92\times$ – $6.24\times$ faster than STRSUB. Output-family analysis further shows Extended B^3 Recall of 0.156 and 0.544 for pairwise-threshold clique mining on two real graphs, quantifying its fragmentation of collectively feasible complete groups.

PVLDB Reference Format:

Tianyi Gao and Hongchao Qin. Maximal Semantic Clique Enumeration in Vector-Attributed Graphs. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

1 INTRODUCTION

Graphs with rich vertex attributes are ubiquitous in modern data management and analytics. Social networks associate users with profile embeddings, knowledge graphs attach entity descriptions to nodes, and biological networks annotate proteins with functional vectors. In all these settings, the attribute space carries semantic meaning that is largely orthogonal to the topological structure: two vertices may be structurally adjacent yet semantically distant, and vice versa. Identifying cohesive subgraphs that are simultaneously structurally dense and semantically coherent is therefore a recurring need across domains.

The maximal clique is a classic structural model of cohesion: every pair of vertices is connected, guaranteeing the highest possible edge density. However, in vector-attributed graphs, a clique may group vertices that are heterogeneous in the attribute space. Conversely, attribute-based community detection methods (e.g., modularity optimization with attribute homogeneity) often sacrifice structural guarantees and produce loosely connected groups. This tension motivates a joint model that enforces both structural completeness and semantic coherence.

The MSC Problem. We study the problem of enumerating *maximal semantic cliques* (MSCs) in a vector-attributed graph. Each vertex carries a real-valued attribute vector; pairwise semantic similarity is measured by cosine similarity. Given a threshold τ , a clique is a τ -semantic clique if its average pairwise similarity is at least τ . An MSC is an inclusion-wise maximal τ -semantic clique: no strict superset is also a τ -semantic clique. The task is to enumerate *all* MSCs of the input graph.

This formulation has two salient features. First, the average constraint is a *global* property of the clique: it depends on all $\binom{|C|}{2}$ pairwise similarities, not on any single edge. This makes it fundamentally different from edge-level semantic constraints (e.g., requiring every edge to exceed a threshold), which are local and monotone. Second, the average is non-monotone over the subset lattice: adding a vertex can raise or lower the average, so a partial clique that currently violates the constraint may still grow into a valid MSC, while one that satisfies it may admit no feasible extension. This non-monotonicity invalidates the structural invariants that underpin the efficiency of classical maximal clique enumeration (MCE) algorithms.

Why Classical MCE Does Not Suffice. The Bron-Kerbosch (BK) algorithm and its modern variants achieve state-of-the-art performance for MCE through two key techniques: *pivot-based pruning* (Tomita et al.) and *degeneracy ordering* (Eppstein et al.). Pivot pruning relies on the fact that structural maximality is closed under supersets: if a vertex u is adjacent to the current pivot p , any maximal clique containing u also contains p , so u can be safely suppressed as an expansion root. Under the average constraint, this closure breaks: a clique containing u but not p may satisfy $\text{avgSim} \geq \tau$ while $\{p, u\}$ does not. Consequently, pivoting is not merely ineffective—it is *incorrect* for MSC enumeration. A direct BK adaptation must therefore forgo pivoting entirely and defer all semantic evaluation to the leaves, degenerating to exhaustive structural enumeration regardless of τ .

Problem formulation. We formalize maximal semantic clique enumeration under an inclusion-wise maximal average-similarity

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

constraint, establish its computational hardness, and identify the non-monotonicity that prevents direct use of structural BK pivoting.

Exact enumeration algorithms. We develop STRSUB, which separates structural MCE from bounded semantic subset enumeration, and MONOSEMMCE, whose canonical-parent relation organizes high-threshold search without visited tables or materialized size layers. Both algorithms preserve the complete MSC output set.

Large-scale evaluation. We evaluate exactness, runtime, memory, scalability, pruning mechanisms, and output semantics on eight vector-attributed graphs under a fixed similarity-quantile protocol. The results identify the operating regions of the two search organizations and quantify their gains over the exact semantic baseline. **Paper Organization.** Section 2 formalizes the problem and establishes hardness. Section 3 presents the baseline SEMBK algorithm and explains why pivoting fails. Section 4 develops STRSUB and MONOSEMMCE. Section 5 describes shared implementation optimizations. Section 6 reports experimental results. Section 7 discusses related work, and Section 8 concludes.

2 PRELIMINARIES

2.1 Notations and Problem Definition

We study a *vector-attributed graph* $\mathcal{G} = (V, E, \mathbf{X})$, where V is the vertex set, $E \subseteq \binom{V}{2}$ is the edge set, and $\mathbf{X} = \{\mathbf{x}_v \in \mathbb{R}^D \mid v \in V\}$ assigns a D -dimensional attribute vector to each vertex.

DEFINITION 1 (SIMILARITY FUNCTION). A similarity function $\text{sim} : \mathbb{R}^D \times \mathbb{R}^D \rightarrow [-1, 1]$ is a symmetric, bounded function measuring the semantic closeness of two attribute vectors. We assume all attribute vectors are ℓ_2 -normalized, so that sim coincides with cosine similarity and its range is naturally $[-1, 1]$.

Throughout this paper we work with normalized vectors and precompute $\text{sim}(u, v)$ for all $(u, v) \in E$. The threshold $\tau \in [-1, 1]$ is drawn from the same range.

DEFINITION 2 (CLIQUE). A set $C \subseteq V$ is a clique in $\mathcal{G}$ if every pair of distinct vertices in C is connected by an edge, i.e., $\{u, v\} \in E$ for all $u \neq v \in C$.

Classical maximal clique enumeration (MCE) seeks cliques that cannot be extended by any additional vertex while preserving the clique property. In vector-attributed graphs, however, structural density alone is insufficient to characterize semantically coherent groups: a clique may consist of vertices that are structurally adjacent yet semantically heterogeneous. To capture both aspects jointly, we introduce two companion quantities.

DEFINITION 3 (PAIRWISE SIMILARITY SUM AND AVERAGE PAIRWISE SIMILARITY). For a clique C of size $k = |C| \geq 2$, the pairwise similarity sum is

$$\text{simSum}(C) = \sum_{\{u, v\} \subseteq C} \text{sim}(u, v),$$

and the average pairwise similarity is

$$\text{avgSim}(C) = \frac{\text{simSum}(C)}{\binom{k}{2}}.$$

DEFINITION 4 (SEMANTIC CLIQUE). Given a threshold $\tau \in [-1, 1]$, a clique C with $|C| \geq 2$ is a τ -semantic clique if $\text{avgSim}(C) \geq \tau$.

For algorithmic purposes, we equivalently track the *semantic surplus*

$$W(C) = \text{simSum}(C) - \tau \binom{k}{2},$$

which is non-negative if and only if $\text{avgSim}(C) \geq \tau$. Working with simSum and W avoids division at every recursive step; avgSim is used only when a human-readable quantity is required.

DEFINITION 5 (MSC). Given $\mathcal{G} = (V, E, \mathbf{X})$ and threshold $\tau \in [-1, 1]$, a τ -semantic clique $C \subseteq V$ is a maximal semantic clique (MSC) if there is no τ -semantic clique $C' \supsetneq C$.

This is inclusion-wise maximality, not merely the absence of a feasible single-vertex extension. The distinction matters because average feasibility is non-monotone. For example, let $\tau = 0.6$ and let four vertices form a clique with $\text{sim}(a, b) = 0.6$, $\text{sim}(a, u) = \text{sim}(b, u) = \text{sim}(a, v) = \text{sim}(b, v) = 0.5$, and $\text{sim}(u, v) = 1$. The feasible clique $\{a, b\}$ has no feasible single-vertex extension, since either three-vertex average is $1.6/3 < 0.6$, but the strict superset $\{a, b, u, v\}$ is feasible with average $3.6/6 = 0.6$. Consequently, algorithms may use the absence of a feasible single extension only to generate candidates; a global containment test is still required for MSC output.

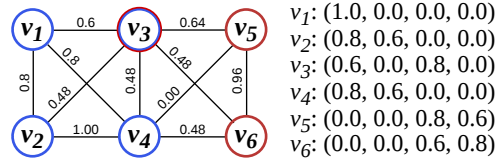


Figure 1: Running example with vector attributes and edge similarities.

EXAMPLE 1. Figure 1 shows a vector-attributed graph with six vertices. Edge labels give the precomputed pairwise similarities. With threshold $\tau = 0.6$, consider the two structural cliques

$$M_1 = \{v_1, v_2, v_3, v_4\}, \quad M_2 = \{v_3, v_4, v_5, v_6\}.$$

Let $C_1 = \{v_1, v_2, v_3, v_4\}$, $C_2 = \{v_3, v_5, v_6\}$, and $C_3 = \{v_3, v_4, v_5, v_6\}$. Their total and average similarities are

$$\begin{aligned} \text{simSum}(C_1) &= 4.16, & \text{avgSim}(C_1) &\approx 0.693 \geq \tau, \\ \text{simSum}(C_2) &= 2.08, & \text{avgSim}(C_2) &\approx 0.693 \geq \tau, \\ \text{simSum}(C_3) &= 2.56, & \text{avgSim}(C_3) &\approx 0.427 < \tau. \end{aligned}$$

Thus C_1 and C_2 are τ -semantic cliques, whereas the structurally valid clique C_3 is not.

Moreover, neither C_1 nor C_2 has a strict superset that is a τ -semantic clique, so both are MSCs.

Problem Statement.

PROBLEM 1 (MSC ENUMERATION). Given a vector-attributed graph $\mathcal{G} = (V, E, \mathbf{X})$ and a threshold $\tau \in [-1, 1]$, enumerate all MSCs of $\mathcal{G}$.

2.2 Challenges

Hardness. MSC enumeration contains classical MCE as a special case. When singleton outputs are excluded and τ does not exceed the minimum similarity on any edge, every structural maximal clique of size at least two satisfies the semantic constraint, so the $\Omega(3^{n/3})$ output lower bound carries over. The associated decision problem is NP-complete.

THEOREM 2.1. *Given $\mathcal{G} = (V, E, X)$, threshold $\tau \in [-1, 1]$, and integer k , deciding whether an MSC of size $\geq k$ exists is NP-complete.*

PROOF. *NP membership.* The decision instance is positive if and only if it contains a τ -semantic clique C of size at least k . The forward implication is immediate. For the reverse implication, the finite family of feasible clique supersets of C contains an inclusion-wise maximal member, whose size is at least $|C| \geq k$. A certificate therefore need only provide such a feasible clique C : verifying all edges and $\text{avgSim}(C) \geq \tau$ takes $O(|C|^2)$ time. It is unnecessary, and under the non-monotone constraint insufficient, to verify maximality by checking only single-vertex extensions.

NP-hardness. We reduce from the CLIQUE decision problem [15], which asks whether $G_0 = (V, E)$ contains a clique of size $\geq k$. Assign $\mathbf{x}_v = \mathbf{e}_1$ to every vertex, so $\text{sim}(u, v) = 1$ for every edge, and set $\tau = 1$. Under this assignment $\text{avgSim}(C) = 1 \geq \tau$ for every clique C , so every structural maximal clique is an MSC and vice versa. Any clique of size $\geq k$ in G_0 is contained in some structural maximal clique of size $\geq k$, which is also an MSC of size $\geq k$. Conversely, any MSC of size $\geq k$ is a clique of size $\geq k$ in G_0 . Hence a size- k MSC exists if and only if G_0 contains a clique of size $\geq k$, completing the reduction. $\square$

The output sets of MSC enumeration and MCE are incomparable: a structural maximal clique with $\text{avgSim}(M) < \tau$ may contain semantically maximal subsets that MCE never generates, while valid MSCs need not be structurally maximal. The output size, and hence the computational cost, is τ -dependent in a non-trivial way that we characterize below.

Output Distribution and Practical Scope. The number of MSCs and their sizes vary substantially with τ . At sufficiently low τ , almost every clique satisfies the average constraint and the output converges toward that of classical MCE. As τ increases, large structural cliques that previously satisfied the constraint cease to do so and are replaced by many smaller semantically feasible sub-cliques, none of which subsumes another. This *fragmentation effect* can inflate the output size by orders of magnitude relative to MCE. At sufficiently high τ , the constraint eliminates most cliques outright and the output shrinks considerably.

The intermediate fragmentation band carries the largest output volume and is therefore inherently time-consuming—any correct algorithm must enumerate all output cliques, so the output size itself imposes a lower bound on runtime. Low τ is appropriate when the goal is to filter out structurally maximal cliques that are semantically too loose; high τ is appropriate when the goal is to surface only the most semantically coherent groups. The exact τ values at which these transitions occur depend on the similarity distribution of the underlying dataset and cannot be fixed a priori. Section 6 therefore

evaluates a fixed grid of per-dataset edge-similarity quantiles rather than choosing absolute thresholds after observing runtimes.

Non-monotonicity and Framework Incompatibility. The core difficulty is that avgSim is non-monotone over the subset lattice. In classical MCE, structural feasibility is monotone and maximality is enforced implicitly by the $P = \emptyset$ invariant at every leaf, so the set of feasible extensions can be characterized solely by the current state of C . Under the average constraint, no such invariant exists: a partial clique with $\text{avgSim}(C) < \tau$ may still grow into a valid MSC, while one with $\text{avgSim}(C) \geq \tau$ may admit no feasible extension. Furthermore, whether a candidate vertex can participate in a valid extension depends on its accumulated similarity to the current partial clique, which evolves throughout the recursion and cannot be captured by structural conditions alone. As a result, the pruning mechanisms based on structural monotonicity lose their foundation, and the pivot strategy no longer applies. Direct adaptations of BK are therefore forced to defer all semantic evaluation to the leaves, where τ appears only in validation rather than guiding the search itself.

3 FRAMEWORK

Framework Overview. We derive a baseline search procedure by adapting the Bron-Kerbosch (BK) recursion to the semantic-clique setting under the average constraint. The standard partition (P, X) remains useful for structural enumeration, but the semantic constraint is defined at the clique level: whether a partial clique can eventually satisfy the threshold depends on avgSim over all vertex pairs within the clique. As a consequence, the pivoting strategy, which relies on local structural properties, is no longer directly applicable. The baseline therefore retains the BK recursion and degeneracy ordering while excluding pivoting, and enforces semantic maximality via a secondary CHECK pass on X at each leaf.

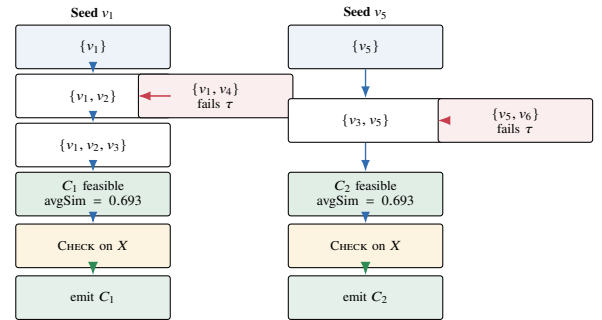


Figure 2: SEMBK traversal on the running example.

[Lines 4–8]: Outer Traversal. The outer loop iterates over vertices in degeneracy order σ . For a seed vertex v , its later neighbors under σ form the candidate set P , and its earlier neighbors are placed in X . This partition guarantees that every structural maximal clique is generated by exactly one seed, because the unique vertex of minimum degeneracy index in any clique serves as its seed and sees all co-members in its P . Since every MSC is contained in at least one structural maximal clique (as established later in Lemma 4.1),

Algorithm 1: Semantic BK with Check

```

1 Input:  $G = (V, E)$ , semantic vectors  $\{x_v\}$ , threshold  $\tau$ 
2 Output: All maximal semantic cliques  $C$  s.t.  $\text{avgSim}(C) \geq \tau$ 
3 Precompute  $\text{sim}(u, v)$  and degeneracy ordering  $\sigma$ ;
4 for each  $v \in \sigma$  in parallel do
5    $P \leftarrow \{u \in N(v) \mid \sigma(u) > \sigma(v)\};$ 
6    $X \leftarrow \{u \in N(v) \mid \sigma(u) < \sigma(v)\};$ 
7   Initialize  $\text{acc}[u] \leftarrow \text{sim}(u, v)$  for all  $u \in N(v)$ ;
8   Search( $C = \{v\}, \text{simSum} = 0, P, X, \text{acc}, \text{mode} = \emptyset, 0$ )

9 Function Search( $C, \text{simSum}, P, X, \text{acc}, \text{mode}, k_{\min}$ ):
10   $k \leftarrow |C|$ ,  $\text{foundLonger} \leftarrow \text{false}$ ;
11  for each  $v \in P$  do
12     $\text{simSum}' \leftarrow \text{simSum} + \text{acc}[v];$ 
13     $P \leftarrow P \setminus \{v\}, P' \leftarrow P \cap N(v), X' \leftarrow X \cap N(v);$ 
14     $\text{acc}[u] \leftarrow \text{acc}[u] + \text{sim}(u, v)$  for  $u \in P' \cup X'$ ;
15     $\text{res} \leftarrow \text{Search}(C \cup \{v\}, \text{simSum}', P', X', \text{acc}, \text{mode}, k_{\min});$ 
16     $\text{acc}[u] \leftarrow \text{acc}[u] - \text{sim}(u, v)$  for  $u \in P' \cup X'$ ;
17    if  $\text{res}$  then
18       $\text{foundLonger} \leftarrow \text{true};$ 
19      if  $\text{mode} = \text{CHECK}$  then return true;
20     $X \leftarrow X \cup \{v\};$ 
21  if  $\neg \text{foundLonger}$  then
22    if  $k \geq 2 \wedge 2 \cdot \text{simSum} / (k(k-1)) < \tau$  then return false;
23    if  $\text{mode} = \text{CHECK}$  then return  $k > k_{\min}$ ;
24    if  $X \neq \emptyset$  then
25      if Search( $C, \text{simSum}, X, \emptyset, \text{acc}, \text{CHECK}, k$ ) then
26        return true;
27    output  $C$ ;
28    return true;
29  return foundLonger;

```

the same partition ensures completeness for the semantic problem without modification.

[Lines 9–29]: Recursive Search. Within each recursive call, the algorithm explores all vertices in P as expansion roots. For each candidate v , it updates simSum and the auxiliary accumulation array acc , then recurses on the reduced candidate and exclusion sets (P', X'). When the recursion reaches a leaf ($\text{foundLonger} = \text{false}$), it first tests the average constraint: if $\text{avgSim}(C) < \tau$, the branch is pruned immediately. Otherwise, semantic maximality must be verified: a clique C with $\text{avgSim}(C) \geq \tau$ and $P = \emptyset$ could still be non-maximal if some vertex in X can extend it to a larger valid clique. The algorithm therefore invokes SEARCH on X in CHECK mode with $k_{\min} = |C|$. In CHECK mode, the recursion explores extensions through X and returns *true* as soon as it finds any valid clique strictly larger than C ; if no such extension exists, C is emitted. The strict size condition $k > k_{\min}$ is necessary because the CHECK recursion may reach sub-cliques of C itself; without recording $k_{\min}$, these would be misidentified as valid extensions.

Why pivoting is not used. In standard BK, a pivot $p \in P \cup X$ guarantees that each structural maximal clique in the current branch contains either p or a vertex in $P \setminus N(p)$. Hence vertices in $P \cap N(p)$ need not become expansion roots. This argument depends

on structural feasibility being preserved when an adjacent pivot is added.

The average constraint invalidates that implication. Consider a complete graph on $\{s, p, u, w\}$ at the BK state $C = \{s\}$, $P = \{p, u, w\}$, and $X = \emptyset$. Let the three similarities within $\{s, u, w\}$ equal 0.9, the three similarities incident to p equal 0, and $\tau = 0.6$. Then $\{s, u, w\}$ is an MSC with average 0.9, whereas its only strict clique superset has average 0.45. Choosing p as the pivot leaves only p in $P \setminus N(p)$, so every explored child contains p and the valid MSC $\{s, u, w\}$ is lost. Pivot-free branching is therefore required for the direct semantic BK baseline. **Complexity.**

THEOREM 3.1 (TIME AND SPACE COMPLEXITY). *Let d_G be the degeneracy of G , let $\mathcal{L}$ be the feasible terminal states reached by the outer search, and let X_C be the candidate set examined by CHECK for $C \in \mathcal{L}$. The running time is $O^*(n^{2^{d_G}} + \sum_{C \in \mathcal{L}} 2^{|X_C|})$, where O^* suppresses polynomial set-intersection costs. A graph-independent upper bound is $O^*(3^n)$. With p workers, the implementation uses $O(m + nD + pn^2)$ worst-case space.*

PROOF. A degeneracy seed has at most d_G later neighbors. Without pivoting, its outer recursion may visit every subset of those neighbors, giving $O^*(n^{2^{d_G}})$ states. For a feasible terminal C , CHECK may independently visit every subset of X_C . The same earlier extension can be reconsidered for several terminals; charging a state to a pair of nested vertex sets gives the safe $O^*(3^n)$ bound. Similarities and sparse adjacency require $O(m + nD)$ storage. Each worker owns an $O(n)$ accumulator, while decreasing candidate arrays retained by a worst-case CHECK stack total $O(n^2)$. $\square$

Design Implication. SEMBK is the direct semantic-search baseline and establishes exact inclusion-wise maximality through its CHECK pass. Its pivot-free branching and repeated extension checks arise from the same property: average similarity couples all pairs in a clique and cannot be summarized by local adjacency alone. Section 4 develops two exact search organizations that expose additional structural or semantic pruning.

4 PROPOSED ALGORITHMS

This section presents two exact algorithms that exploit the global structure of the average constraint from different directions.

STRSUB (Section 4.1) resolves the pivot incompatibility by decoupling structural and semantic search. The pivot is applied without modification to a τ -agnostic structural phase, restoring the $O(3^{n/3})$ BK guarantee; τ is then enforced in a subsequent intra-clique enumeration phase. Complementary sorted-edge and vertex-contribution upper bounds introduced in Phase 2 skip subset-size layers that cannot satisfy τ . Its cost is governed by structural MCE and by the sizes of the resulting host cliques.

MONOSEMMCE (Section 4.2) departs from the BK framework entirely. Motivated by the observation that STRSUB, despite its pruning improvements, still processes all structural maximal cliques irrespective of their semantic quality, MONOSEMMCE introduces a canonical reverse-search strategy in which τ drives enumeration from the first expansion step. A deterministic feasible parent gives each feasible clique one generation path, while depth-first traversal avoids storing a complete size layer. The method is particularly

effective at high τ , where few candidate extensions pass the surplus test.

Graph size and structural clique growth determine the crossover between the two methods. We therefore evaluate them on the same quantile grid: STRSUB provides broad threshold coverage, while MONOSEMCE specializes the search for selective high- τ workloads.

4.1 STRSUB: Structural Decomposition and Subset Enumeration

Structural–Semantic Decoupling and Containment. The validity of the Bron–Kerbosch pivot rests on a structural invariant: every structural maximal clique must contain at least one vertex from $P \setminus N(u)$ for any pivot $u \in P \cup X$. This invariant is violated when the average-similarity constraint is incorporated directly into the branching procedure, because avgSim is a *global* property of a candidate set and does not decompose along the local neighborhood structure used to justify the pivot. Consequently, the semantic variant SEMBK must forgo pivoting entirely, which undermines the $O(3^{n/3})$ worst-case guarantee of the standard algorithm [26].

Figure 3 separates the two host-clique outcomes. Gray marks structural or global processing, green marks accepted feasible states, and amber marks the bounded search triggered by an infeasible host. Thus M_1 is accepted as C_1 , whereas subset enumeration within M_2 recovers C_2 .

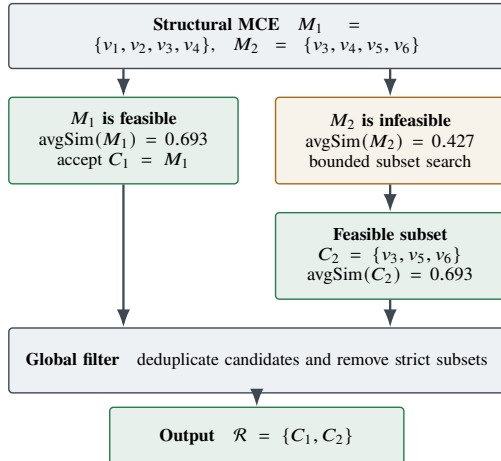


Figure 3: STRSUB execution on the running example.

The key insight behind STRSUB is that the structural and semantic layers of the problem can be *decoupled*. Specifically, the Tomita pivot can be applied without modification to the purely structural problem of enumerating maximal cliques in G , because that problem carries no average-similarity constraint. The threshold τ is then enforced in a second, independent pass confined to each structural maximal clique. The theoretical basis for this decoupling is the following containment lemma.

LEMMA 4.1 (STRUCTURAL SUPERSET OF MSCs). *Every MSC is contained in at least one structural maximal clique of G . That is, for*

Algorithm 2: STRSUB: Structural Decomposition and Subset Enumeration

Input: $G = (V, E)$, similarities $\{\text{sim}(u, v)\}_{(u,v) \in E}$, threshold τ
Output: All MSCs $\mathcal{R}$

```

1  $\mathcal{F} \leftarrow \emptyset$ ;
  /* Pipelined structural MCE and semantic processing */
2 Build degeneracy ordering  $\sigma$ ;
3 for each  $v \in \sigma$  in parallel do
4    $P \leftarrow \{u \in N(v) \mid \sigma(u) > \sigma(v)\}$ ;
5    $X \leftarrow \{u \in N(v) \mid \sigma(u) < \sigma(v)\}$ ;
6   Run BK( $\{v\}, P, X$ ) with the following callback at each leaf  $M$ ;
  /* Callback: intra-clique semantic enumeration */
7 for each structural leaf  $M$  produced by a worker do
8   if avgSim( $M$ )  $\geq \tau$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{M\}$ ; continue;
  /* Admissible layer bounds */
9   Sort edges of  $M$  by similarity descending;
10   $k^* \leftarrow \max\{k \in [2, |M| - 1] \mid e_k \geq \tau\}$ ;
11  if  $k^*$  does not exist then continue;
12   $lmax \leftarrow \emptyset$ ;  $umask \leftarrow \emptyset$ ;
13  for  $k \leftarrow k^*$  to 2 do
14    Compute the vertex-contribution bound  $\bar{q}_k$  for size  $k$ ;
15    if  $\bar{q}_k < \tau$  then continue;
16    for each size- $k$  subset  $S \subseteq M$  do
17      if  $(S \& umask) = S$  and  $\exists T \in lmax : S \subseteq T$  then
18        continue;
19      if avgSim( $S$ )  $\geq \tau$  then
20         $lmax \leftarrow lmax \cup \{S\}$ ;
21         $umask \leftarrow umask \mid S$ ;
22         $\mathcal{F} \leftarrow \mathcal{F} \cup \{S\}$ ;
  /* Phase 3: Exact global filter */
23 Sort  $\mathcal{F}$  by  $|\cdot|$  descending; remove exact duplicates;
24 Initialize an empty inverted index  $inv : v \mapsto \emptyset$ ;
25  $\mathcal{R} \leftarrow \emptyset$ ;
26 for each equal-size group  $\mathcal{G} \subseteq \mathcal{F}$  in descending size do
27   for each  $S \in \mathcal{G}$  in parallel do
28      $r \leftarrow \arg \min_{v \in S} |inv[v]|$ ;
29     Keep  $S$  iff no  $T$  referenced by  $inv[r]$  satisfies  $S \subseteq T$ ;
30 Insert every kept  $S \in \mathcal{G}$  into  $\mathcal{R}$  and  $inv$ ;
31 return  $\mathcal{R}$ ;

```

every MSC C , there exists a structural maximal clique M such that $C \subseteq M$.

PROOF. By definition, every MSC is a clique of G . Let C be an arbitrary MSC, and consider the family

$$\mathcal{F} = \{D \subseteq V \mid C \subseteq D \text{ and } D \text{ is a clique in } G\}.$$

Since $C \in \mathcal{F}$, the family is nonempty. Because V is finite, $\mathcal{F}$ is finite, so we may choose a set $M \in \mathcal{F}$ that is maximal under inclusion. By construction, M is a clique containing C , and no vertex $w \in V \setminus M$ extends M to a larger clique; hence M is a structural maximal clique of G , and $C \subseteq M$. $\square$

Lemma 4.1 establishes that the complete collection of MSCs can be recovered by (i) enumerating all structural maximal cliques, and

(ii) searching within each structural maximal clique for semantically feasible maximal subsets. Step (i) is a purely structural problem to which the full Tomita pivot applies, restoring the $O(3^{n/3})$ worst-case bound of standard BK. The threshold τ enters only in step (ii), where the search space is bounded by the size of a single structural clique. Algorithm 2 implements this strategy in three phases.

[Lines 1–6]: Phase 1 – Structural Maximal Clique Enumeration. Phase 1 enumerates all structural maximal cliques of G using the standard BK algorithm equipped with the Tomita pivot [26] and degeneracy ordering [7]. Vertices are processed in degeneracy order σ ; for each seed vertex v , the candidate set P and exclusion set X are initialized to the forward and backward neighborhoods of v in σ , respectively, and recursive BK is invoked on the triple $(\{v\}, P, X)$. Seeds are dispatched in parallel via OpenMP dynamic scheduling. Crucially, this phase is entirely τ -agnostic: the threshold plays no role in branching decisions, so the full pivot strategy and its associated complexity bound are preserved. The implementation does not materialize the set of all structural maximal cliques: when BK reaches a leaf M , the same worker immediately invokes the Phase 2 callback and retains only its thread-local semantic candidates.

[Lines 7–20]: Phase 2 – Intra-Clique Semantic Enumeration. For each structural maximal clique M of size m produced by Phase 1, Phase 2 enumerates all semantically maximal subsets of M with respect to τ . Subsets of M are generated in descending order of cardinality using native-word bitmasks when possible and dynamic bitsets for larger host cliques; no mask-width limit is imposed. Pairwise similarities are precomputed into an $m \times m$ local matrix to amortize repeated avgSim evaluations.

A *local maximality filter* prunes redundant candidates during enumeration. The filter maintains a set $lmax$ of valid subsets discovered so far within M , together with a union mask $umask = \bigcup_{T \in lmax} T$ that provides a constant-time necessary condition for subsumption. A candidate subset S is skipped if $S \subseteq T$ for some $T \in lmax$, verified first via the bitmask test $(S \& umask) = S$ and then by a linear scan over $lmax$. This filter is correct because, if $S \subseteq T$ and $\text{avgSim}(T) \geq \tau$, then S cannot be semantically maximal in G regardless of its own average similarity. Structural leaves are processed by their owning workers. The candidate size layers are further reduced by the admissible bounds described below.

[Lines 21–28]: Phase 3 – Global Deduplication. Because the same MSC may be structurally contained in multiple distinct maximal cliques, Phase 2 may emit duplicate or subsumed candidates across cliques. Phase 3 consolidates these candidates into the final result set $\mathcal{R}$.

The phase proceeds in two steps. First, candidates are sorted by decreasing size with a lexicographic tie break, and exact duplicates are removed in $O(N \log N)$ time, where $N = |\mathcal{F}|$. Second, equal-size groups are processed in that order. The inverted index $inv : v \mapsto \{T \in \mathcal{R} \mid v \in T\}$ contains only retained strict supersets of the current group. For each candidate S , the algorithm identifies its *rarest vertex* $r = \arg \min_{v \in S} |inv[v]|$ and scans only $inv[r]$; a sorted-merge traversal determines in $O(|S| + |T|)$ time whether $S \subseteq T$. Queries within one group read the same frozen index and run in parallel, after which the retained candidates are inserted. Equal-size sets cannot strictly contain one another. Moreover, if a discarded larger candidate contains S , transitivity guarantees that

a retained candidate containing both will be indexed. The grouped filter is therefore equivalent to testing all larger candidates while avoiding their indexing and synchronization costs.

Admissible Layer Pruning. Phase 2 as described iterates over all subset sizes from $|M| - 1$ down to 2. A key observation allows the starting size to be reduced: the best achievable avgSim for any size- k subset of M is bounded above by the mean of the $\binom{k}{2}$ largest pairwise similarities in M .

PROPERTY 4.1 (SORTED-EDGE UPPER BOUND). *Let $e_1 \geq e_2 \geq \dots \geq e_{\binom{m}{2}}$ be the pairwise similarities of M sorted in descending order. For any size- k subset $S \subseteq M$,*

$$\text{avgSim}(S) \leq \bar{e}_k := \frac{1}{\binom{k}{2}} \sum_{i=1}^{\binom{k}{2}} e_i.$$

PROOF. Any size- k subset $S \subseteq M$ uses exactly $\binom{k}{2}$ pairs, each contributing at most the corresponding value in the sorted sequence $e_1 \geq e_2 \geq \dots$; hence $\text{simSum}(S) \leq \sum_{i=1}^{\binom{k}{2}} e_i$, and dividing both sides by $\binom{k}{2}$ gives the result. $\square$

The sorted-edge bound ignores that its largest edges may be concentrated on too few vertices to coexist in one size- k subset. A complementary bound tracks which vertices support those similarities. For each $v \in M$, let $q_v(k)$ be the sum of the $k - 1$ largest similarities between v and vertices in $M \setminus \{v\}$. Let $q_{(1)}(k) \geq \dots \geq q_{(m)}(k)$ denote these values in descending order.

PROPERTY 4.2 (VERTEX-CONTRIBUTION UPPER BOUND). *For any size- k subset $S \subseteq M$,*

$$\text{avgSim}(S) \leq \bar{q}_k := \frac{1}{k(k-1)} \sum_{i=1}^k q_{(i)}(k).$$

PROOF. For every $v \in S$, the sum of its similarities to the other vertices of S uses $k - 1$ vertices from $M \setminus \{v\}$ and is therefore at most $q_v(k)$. Summing over S counts each internal pair twice, so

$$2\text{simSum}(S) \leq \sum_{v \in S} q_v(k) \leq \sum_{i=1}^k q_{(i)}(k).$$

Division by $k(k-1)$ gives the claimed average-similarity bound. $\square$

Note that $\bar{e}_k$ is non-increasing in k : a larger subset requires more edges, drawing further into the sorted list and lowering the mean. Before enumeration, all pairwise similarities of M are sorted once in $O(m^2 \log m)$ and prefix means $\bar{e}_k$ are computed for all k . The algorithm identifies

$$k^* = \max\{k \in [2, |M| - 1] \mid \bar{e}_k \geq \tau\},$$

and begins enumeration at $k = k^*$ rather than $|M| - 1$. Because $\bar{e}_k$ is non-increasing, all sizes $k > k^*$ satisfy $\bar{e}_k < \tau$, certifying that no subset of those sizes can meet the constraint; they are skipped entirely. If no such k^* exists, the whole clique M is skipped. For every remaining size k , the algorithm also skips the layer when $\bar{q}_k < \tau$. The two tests are independently admissible, so using their minimum cannot remove a feasible subset. Computing all per-vertex similarity lists and prefix sums costs $O(m^2 \log m)$, matching the sorting order of the Top-Edge preprocessing.

Characteristics and Applicable Regime. STRSUB applies Tomita pivoting during structural MCE, whose cost is $O(d_G n 3^{d_G/3})$ and independent of τ . At low thresholds, large feasible subsets are identified early within each structural host. At higher thresholds, the Top-Edge and vertex-contribution bounds skip infeasible size layers. Structural MCE remains a fixed front-end cost, which motivates the canonical search developed next for selective thresholds.

Complexity.

THEOREM 4.1 (TIME COMPLEXITY OF STRSUB). *For each structural maximal clique M , let Q_M denote the number of exact local containment probes and let $b_M = \lceil |M|/w \rceil$ be its bitmap length in machine words. STRSUB runs in*

$$O\left(d_G n 3^{d_G/3} + \sum_{M \in \mathcal{M}} \left(2^{|M|} |M|^2 + Q_M b_M\right) + KN \log N + K \sum_{S \in \mathcal{F}} L_{\min}(S)\right)$$

time, where d_G is the degeneracy of G , $N = |\mathcal{F}|$ is the number of candidates before global deduplication, $K = \max_{S \in \mathcal{F}} |S|$, and $L_{\min}(S) = \min_{v \in S} |\text{inv}[v]|$.

PROOF. The structural phase uses degeneracy-oriented Tomita BK [7], with cost $O(d_G n 3^{d_G/3})$. For a host M of size h , subset enumeration visits at most 2^h subsets and evaluates each from the cached similarity matrix in $O(h^2)$ time. Exact local maximality performs Q_M bitmap containment probes of b_M words each; keeping this term explicit also covers instances with many feasible local subsets. The final filter sorts N candidates in $O(KN \log N)$ time. It then scans the shortest inverted list for each S , with at most $L_{\min}(S)$ sorted-set containment checks of $O(K)$ time. $\square$

Remark. Without pruning, the enumeration term is exponential in each structural host, and Q_M is output sensitive. The admissible bounds replace $2^{|M|}$ by $\sum_{k=2}^{|M|} \binom{|M|}{k}$ when all larger layers are certified infeasible; local containment reduces the candidates reaching the global filter without changing the output set.

4.2 MONOSEMMCE: High- τ Canonical Reverse Search

Motivation and Core Insight. Even with the Top-Edge Layer Pruning, STRSUB is bound to Phase 1: it enumerates all structural maximal cliques before any semantic evaluation takes place. In the high- τ regime, the vast majority of structural cliques may be semantically barren, yet each still contributes Phase 1 cost. More fundamentally, the BK framework offers no mechanism to incorporate τ into branching decisions—the threshold appears only in validation, never in the search logic itself. This structural overhead motivates abandoning the BK skeleton entirely.

The obstacle to integrating τ directly into search is the non-monotonicity of avgSim : an infeasible clique can have a feasible strict superset. A search that retains only feasible states is therefore complete only if every feasible target has a feasible construction path. Theorem 4.2 supplies such a path through the deletion rule defined next.

For a vertex v in a clique C , define its *similarity contribution* as $a_C(v) = \sum_{u \in C \setminus \{v\}} \text{sim}(u, v)$: the sum of its similarities to the other vertices in C . We delete a vertex with minimum similarity contribution, breaking ties by vertex identifier. The theorem below

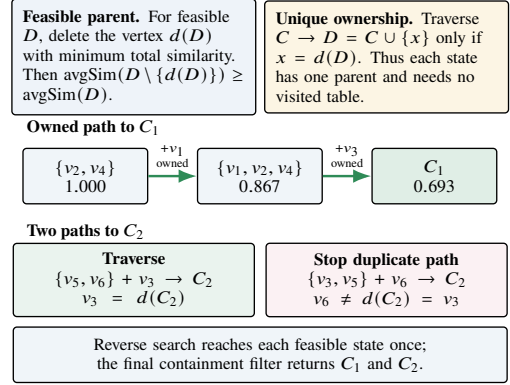


Figure 4: Canonical ownership on the running example.

proves that the remaining clique is feasible. This deterministic rule gives every feasible clique a unique parent, called its *canonical parent*. Reverse search accepts an extension only when its parent under this rule is the current clique. Thus every feasible clique is generated once without a visited table or per-layer deduplication.

The implementation traverses each canonical tree depth-first and parallelizes its feasible edge roots. It rejects an extension whenever its updated surplus is negative. Unlike the previous level-synchronous algorithm, it stores no complete size layer; working memory consists of the graph representation, thread-local accumulators, DFS scratch space, and output candidates. A final containment filter is still necessary, because absence of a feasible single extension is weaker than inclusion-wise maximality.

The Monotone Path Property.

THEOREM 4.2 (FEASIBLE MINIMUM-CONTRIBUTION DELETION). *Let C be a τ -semantic clique of size $k \geq 3$, with the similarity contribution $a_C(v)$ defined above. If $d(C)$ minimizes the pair $(a_C(v), v)$ lexicographically, then $C \setminus \{d(C)\}$ is a τ -semantic clique. Repeated application of this rule therefore reaches a feasible edge, and reversing these deletions gives a non-increasing-average insertion path.*

PROOF. Each pair is counted once at each endpoint. The minimum contribution therefore satisfies $a_C(d(C)) \leq 2\text{simSum}(C)/k$. Deleting $d(C)$ leaves similarity sum at least $\text{simSum}(C)(k-2)/k$, and hence

$$\text{avgSim}(C \setminus \{d(C)\}) \geq \frac{\text{simSum}(C)(k-2)/k}{\binom{k-1}{2}} = \text{avgSim}(C) \geq \tau.$$

Deterministic tie-breaking makes $d(C)$ unique. Applying the same argument until two vertices remain preserves feasibility at every deletion; reversing the sequence proves the insertion-path claim. $\square$

Figure 4 applies this rule to the running example. The green paths insert the canonical deletion of each child, while the red path reaches C_2 through a non-owning insertion and stops before recursion.

Feasible Edge Roots. Every feasible clique of size at least two reaches a feasible edge under the deterministic deletion rule, so Algorithm 3 starts one tree for each edge with similarity at least τ . A

Algorithm 3: MONOSEMMCE: Canonical Reverse Search

```

1 Input:  $G = (V, E)$ , semantic vectors  $\{x_v\}$ , threshold  $\tau$ 
2 Output: All MSCs  $\mathcal{R}$ 
3 Precompute  $\text{sim}(u, v)$  for all  $(u, v) \in E$ ;
4  $\mathcal{F} \leftarrow \emptyset$ ;
5 for each  $(u, v) \in E$  with  $u < v$  and  $\text{sim}(u, v) \geq \tau$  in parallel do
6    $C \leftarrow \{u, v\}$ ;  $S \leftarrow \text{sim}(u, v)$ ;
7    $P \leftarrow N(u) \cap N(v)$ ;
8    $\text{acc}[w] \leftarrow \text{sim}(w, u) + \text{sim}(w, v)$  for  $w \in P$ ;
9    $a_C(u) = a_C(v) \leftarrow \text{sim}(u, v)$ ;
10   $\text{Search}(C, S, P, \text{acc}, a_C)$ ;
11 Apply Algorithm 2's grouped strict-containment filter to  $\mathcal{F}$ ,
   obtaining  $\mathcal{R}$ ;
12 return  $\mathcal{R}$ ;

13 Function  $\text{Search}(C, S, P, \text{acc}, a_C)$ :
14    $k \leftarrow |C|$ ;  $W \leftarrow S - \tau \binom{k}{2}$ ;
15    $\text{hasValid} \leftarrow \text{false}$ ;
16   for each  $v \in P$  do
17      $\delta \leftarrow \text{acc}[v]$ ;
18     if  $W + \delta - \tau k < 0$  then continue;
19      $\text{hasValid} \leftarrow \text{true}$ ;
20      $D \leftarrow C \cup \{v\}$ ;
21     Update  $a_D$  from  $a_C$  and  $\text{sim}(v, u)$  for  $u \in C$ ;
22     if  $v \neq \arg \min_{x \in D} (a_D(x), x)$  then continue;
23      $P' \leftarrow P \cap N(v)$ ;
24      $\text{acc}'[w] \leftarrow \text{acc}[w] + \text{sim}(w, v)$  for  $w \in P'$ ;
25      $\text{Search}(D, S + \delta, P', \text{acc}', a_D)$ ;
26   if not  $\text{hasValid}$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{C\}$ ;

```

state stores the clique C , its pairwise sum S , the common-neighbor set P , candidate accumulators acc , and the member contributions a_C . No BK exclusion set is required: uniqueness follows from the canonical-parent test rather than from an insertion order.

Surplus and Canonical-Child Tests. For $k = |C|$ and $\delta = \text{acc}[v]$, the extension satisfies

$$W(C \cup \{v\}) = W(C) + \delta - \tau k.$$

If this value is negative, the child itself is infeasible and cannot lie on the canonical path of any feasible answer, so it is safely skipped. For a feasible child $D = C \cup \{v\}$, the algorithm updates every member's similarity contribution and accepts D only if $v = d(D)$. Equivalently, the canonical parent of D must be exactly C . This test removes alternative insertion paths before recursion rather than after state materialization. The flag hasValid is set before the ownership test because it records the existence of any feasible one-vertex extension, not the existence of an owned child. A state with such an extension cannot itself be inclusion-wise maximal; the extension is generated from its unique canonical parent.

The path property is used to assign unique parent ownership rather than to order candidates. The algorithm tests child feasibility directly and then applies the minimum-contribution deletion rule. This removes duplicate paths before recursion without a candidate-order cutoff. Accumulator values are restored from saved values

when backtracking, so the result does not depend on reversing a sequence of floating-point additions.

Inclusion-Wise Maximality. A state with no feasible single-vertex extension is added to $\mathcal{F}$, but is not yet declared an MSC. If such a candidate C is not inclusion-wise maximal, the finite family of feasible clique supersets of C contains an inclusion-wise maximal member M . The canonical search enumerates M exactly once, and M has no feasible single extension, so it also belongs to $\mathcal{F}$. The final grouped inverted-index containment filter consequently removes C . Conversely, an MSC is not contained in any other feasible candidate and is retained.

THEOREM 4.3 (ENUMERATION CORRECTNESS). *Algorithm 3 generates every τ -semantic clique of size at least two exactly once and returns exactly all MSCs.*

PROOF. Existence follows by reversing the repeated deterministic deletions in Theorem 4.2; every prefix is feasible and therefore passes the surplus test. Uniqueness follows because every non-root feasible clique has the single parent $C \setminus \{d(C)\}$, and only that parent accepts it as a child. The preceding containment argument proves that the final filter retains precisely the inclusion-wise maximal feasible cliques. $\square$

Complexity.

THEOREM 4.4 (TIME AND SPACE COMPLEXITY OF MONOSEMMCE). *Let F_τ be the number of feasible cliques of size at least two, K the maximum clique size, Δ the maximum degree, T the number of workers, and $N = |\mathcal{F}|$ the number of terminal candidates. A conservative bound for canonical search is*

$$O\left(F_\tau \Delta (\Delta + K) \log \Delta + KN \log N + K \sum_{S \in \mathcal{F}} L_{\min}(S)\right),$$

with $O(m + T(n + K\Delta) + NK)$ words of space in addition to vector storage.

PROOF. Canonical ownership bounds accepted states by F_τ . A state tests at most Δ candidates. For each candidate, common-neighbor intersection and accumulator maintenance cost $O(\Delta)$ conservatively, while updating and comparing at most K member contributions uses cached similarities with $O(\log \Delta)$ lookup. This is bounded by $O(\Delta(\Delta + K) \log \Delta)$ per accepted state. The final filter has the same $O(KN \log N + K \sum_S L_{\min}(S))$ cost as in STRSUB.

Sparse similarity adjacency and neighbor bitmaps use $O(m)$ words. Each worker owns an $O(n)$ generation-tagged accumulator and $O(K\Delta)$ DFS scratch space. Terminal candidates occupy $O(NK)$ words; no feasible-size layer or visited-state table is materialized. $\square$

Remark. Increasing τ contracts the feasible-state family F_τ , which gives MONOSEMMCE its selective-threshold operating region. STRSUB instead pays the structural MCE cost and is less sensitive to F_τ .

5 IMPLEMENTATION OPTIMIZATIONS

The following optimizations are applied to MONOSEMMCE and STRSUB without affecting correctness. They address three categories of bottlenecks: parallel load balance, repeated computation in the

expansion inner loop, and memory access cost for neighbor intersection and similarity lookup.

Parallel Decomposition and Load Balance. STRSUB Phase 1 parallelizes BK over seed vertices under degeneracy ordering: each seed v spawns an independent subproblem over its forward neighborhood (P_v, X_v) , and distinct seeds share no mutable state and require no synchronization. Phase 2 normally processes each structural maximal clique with thread-local candidate lists, but a few large host cliques can dominate this work. For such a clique, each fixed-cardinality layer is partitioned by its smallest selected vertex. These partitions are disjoint and may run concurrently because distinct equal-size subsets cannot strictly contain one another. A barrier merges the feasible subsets before the next smaller layer, preserving all local-containment decisions while exposing parallelism inside a single expensive host clique. Thread-local candidates are merged only after processing, avoiding lock contention on the output.

MONOSEMMCE assigns each feasible edge root to one canonical tree. Distinct trees contain disjoint feasible-clique states, so roots are dispatched with dynamic OpenMP scheduling and require no shared visited table. Each worker performs DFS with thread-local accumulators, scratch buffers, statistics, and candidate output; only the final merge and containment filter synchronize globally.

Incremental Candidate Maintenance. When expanding state $(C, \text{simSum}, P, \text{acc})$ by vertex v , the child state requires an updated candidate set $P' = P \cap N(v)$ and an updated accumulator acc' . A naïve implementation recomputes acc' from scratch at cost $O(|P'| \cdot |C|)$ per expansion.

Since $\text{acc}[w] = \sum_{u \in C} \text{sim}(w, u)$, the child accumulator satisfies

$$\text{acc}'[w] = \text{acc}[w] + \text{sim}(w, v) \quad \forall w \in P',$$

reducing the update to $O(|P'|)$ regardless of clique depth. The DFS records each previous accumulator value and restores it exactly after the recursive call. Reversing an update by subtracting the same floating-point similarity is not equivalent: rounding can accumulate across sibling branches and change a threshold decision. Candidate ordering has no correctness role in canonical search, so the standard implementation performs no per-state sort.

Sparse Blocked Bitmap for Neighbor Intersection. Candidate set intersection $P' = P \cap N(v)$ is the innermost operation of every expansion. We represent each adjacency list as a *sparse blocked bitmap*: vertices are indexed by $\lfloor v/64 \rfloor$, and only non-empty 64-bit blocks are stored as (index, word) pairs in sorted order. Intersection of two rows executes a merge-style scan over their block lists, applying bitwise AND only to matching block indices and extracting set bits via `ctz`. The cost is $O(b_u + b_v)$ where b_u, b_v count non-empty blocks in each row, compared to $O(n/64)$ for a dense bitmap.

This advantage compounds with search depth: at layer ℓ , every candidate $w \in P$ is a common neighbor of all $\ell+2$ clique members, so $|P|$ decreases with ℓ and each row’s active block count shrinks accordingly. The per-intersection cost therefore decreases with depth.

Similarity Computation with Reorder-Aware Caching. All pairwise similarities are precomputed once and stored in sorted adjacency lists for $O(\log \deg(u))$ lookup via binary search. Inner products use 8-wide AVX2 SIMD over the float32 vector store. Edge

Dataset	$ V $	$ E $	D	Attributes
FB15K-237	14,541	248,611	384	natural
GitHub-Social	37,700	289,003	4,005	natural
ogbn-arxiv	169,343	1,157,799	128	natural
email-Enron	36,692	183,831	256	generated
sc-nasasrb	54,870	1,311,227	256	generated
sc-pkustk11	87,804	2,565,054	256	generated
sc-pwtk	217,891	5,653,221	256	generated
sc-lldoor	909,537	20,770,807	256	generated

Table 1: [RQ1–RQ5] Eight public graphs cover natural and controlled vector attributes.

similarities remain float32 to control memory traffic, while similarity sums, per-candidate accumulators, surpluses, and threshold products are maintained in double precision. Every feasibility test compares these double-precision quantities directly with an inclusive boundary; no algorithm-specific epsilon is used.

To reduce cache miss cost on repeated lookups, vertices are reordered by a degree-descending BFS before vectors are stored contiguously in memory. A vertex u with degree d_u participates in $O(d_u)$ expansions as a candidate across the search; placing high-degree vertices at low memory offsets concentrates the most frequently accessed vectors into a small number of cache lines, reducing cold misses on the hot working set.

6 EXPERIMENTS

We organize the evaluation around five research questions. RQ1 evaluates exact end-to-end performance; RQ2 identifies threshold and scale regimes; RQ3 isolates parallel and pruning mechanisms; RQ4 measures how the MSC output space changes with τ ; and RQ5 examines semantic quality against structurally defined alternatives.

6.1 Experimental Setup

Methods and comparison scope. STRUCTBK is a Tomita-pivot structural MCE implementation and supplies only a threshold-independent structural-time reference. SEMBK is the exact direct baseline: it uses the same MSC definition, threshold, similarity values, and strict-containment filter as STRSUB and MONOSEMMCE. Consequently, runtime speedups are claimed only among these three exact MSC methods. PAIRSIM-MCE keeps edges whose end-point similarity reaches τ and then runs structural MCE. FastQC enumerates maximal quasi-cliques under a degree-density constraint [33], while FaPlex enumerates maximal 2-plexes that may omit one incident edge per vertex [35]. These methods define different pattern families and are used only for output-quality comparisons. **Datasets.** The common grid contains three graphs with natural attributes and five public graphs with reproducibly generated attributes. FB15K-237 descriptions use all-MiniLM-L6-v2 embeddings; GitHub-Social uses its official 4,005-dimensional developer features; and ogbn-arxiv retains its official 128-dimensional node features [12, 27]. For email-Enron and the four SuiteSparse graphs [5, 17], PCG64 draws 256 coordinates independently from $U[-1, 1)$ under fixed dataset seeds, followed by ℓ_2 normalization. Graph structure is never synthesized.

Protocol. Experiments run on an AMD Ryzen 9 7945HX workstation with 16 physical cores, 32 hardware threads, and 32 GiB memory under Windows 11 and MSYS2 GCC 13.2. Each process is limited to 16 GiB and 32 OpenMP threads. The uniform timeout is 1,200 seconds. Completed runtime configurations are repeated three times and reported by the median; a timeout is recorded after one complete limit window. Runtime binaries contain no statistics counters.

Absolute cosine values are not comparable across vector sources. We therefore derive thresholds from the edge-similarity distribution of each dataset. The common grid fixes q_{20} , q_{80} , and q_{99} before execution; sensitivity experiments use q_5 , q_{20} , q_{50} , q_{70} , q_{80} , q_{90} , q_{95} , q_{97} , and q_{99} . All exact methods receive the same stored float32 edge similarities and evaluate inclusive feasibility using double-precision sums.

6.2 RQ1: Exactness and End-to-End Efficiency

This experiment answers RQ1 by comparing complete MSC output sets and total runtime under the common grid. Exhaustive oracles first test 225,000 vertex-bound instances and 40,001 canonical-parent instances, including negative similarities, equality at τ , and tied minimum contributions. On real graphs, jointly completed methods agree on normalized clique sets, counts, and size histograms. In particular, all three exact methods return 263,721, 71,118, and 2,636 MSCs on GitHub-Social at q_{20} , q_{80} , and q_{99} , respectively; STRSUB and MONOSEMMCE also agree on every jointly completed row of Table 2.

Table 2 shows that STRSUB completes 22 of 24 workloads and every q_{20} row, while MONOSEMMCE resolves the two intermediate rows where host-clique subset enumeration reaches TLE. At q_{99} , canonical search is faster on all eight graphs, with speedups from 2.92 $\times$ on sc-lldoor to 6.24 $\times$ on email-Enron. SEMBK completes all nine natural-attribute rows except FB15K-237, but its pivot-free search is slower than the best specialized method on every completed row. The gray STRUCTBK column is reported only to separate structural enumeration cost from semantic processing.

6.3 RQ2: Threshold and Scale Regimes

This experiment answers RQ2 by varying selectivity independently of graph size. Figure 5 uses the full fixed quantile grid. On ogbn-arxiv, both specialized methods complete the sweep and cross near q_{95} ; on sc-lldoor, STRSUB covers low and intermediate thresholds, while MONOSEMMCE becomes dominant once the feasible canonical tree contracts. TLE points are shown at the fixed limit and are not interpolated.

Figure 6 repeats the comparison on nested induced ogbn-arxiv graphs whose vertices follow one fixed degree order. The crossover remains at q_{95} from 25K vertices through the full graph. At q_{99} on the full graph, STRSUB and MONOSEMMCE take 0.707 and 0.200 seconds and return the same 10,261 MSCs. The stable boundary links the regime change to semantic selectivity rather than one graph size.

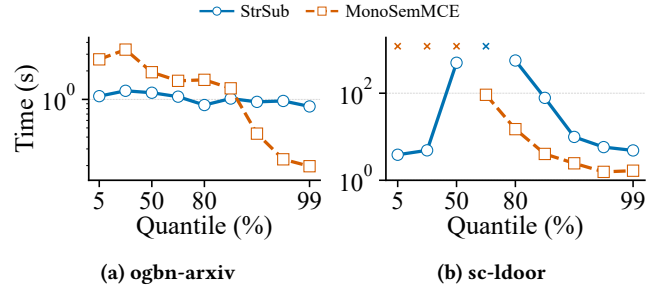


Figure 5: [RQ2] Threshold sweeps reveal complementary runtime regimes.

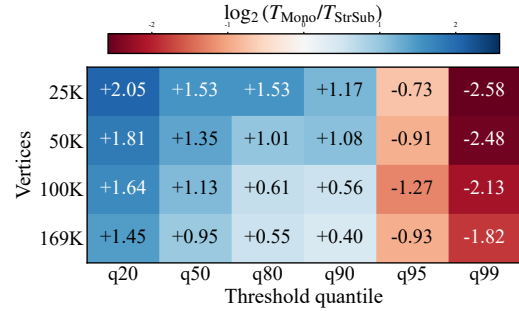


Figure 6: [RQ2] The MONOSEMMCE high-threshold region persists across four nested graph sizes.

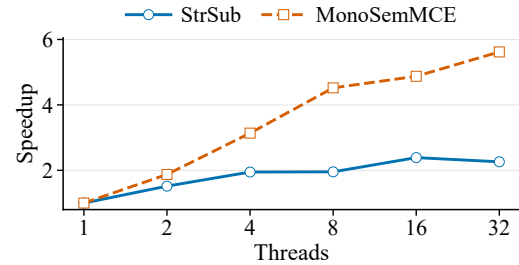


Figure 7: [RQ3] Independent root scheduling yields parallel gains for both specialized searches.

6.4 RQ3: Parallelism and Search Mechanisms

This experiment answers RQ3 by connecting parallel speedup to measured search work. Figure 7 reports one representative workload for each specialized algorithm. Dynamic edge-root scheduling gives MONOSEMMCE 5.62 $\times$ speedup at 32 threads. Fixed-size subset partitioning gives STRSUB 2.39 $\times$ at 16 threads and 2.26 $\times$ at 32 threads. Separately collected phase counters attribute 98.88% of its 32-thread work to host-clique subset processing, while the final global filter takes 65 ms; the remaining saturation is therefore caused by unequal host-clique tasks.

Figure 8 relates runtime regimes to work counters collected outside timing runs. On sc-nasasrb at q_{99} , Top-Edge pruning reduces

Dataset	q	MSCs	STRUCTBK structure	SEMBK	STRSUB	MONOSEMCE
FB15K-237	20	206,470	0.103	TLE	0.221	TLE
	80	76,106		TLE	9.132	TLE
	99	7,476		TLE	45.234	10.091
GitHub-Social	20	263,721	0.110	55.269	0.327	29.315
	80	71,118		44.161	0.314	20.551
	99	2,636		38.213	0.391	0.118
ogbn-arxiv	20	925,239	0.309	2.783	1.233	3.357
	80	226,936		1.206	0.872	1.610
	99	10,261		1.318	0.843	0.197
email-Enron	20	223,703	0.090	18.330	0.259	16.739
	80	60,352		21.369	1.266	0.083
	99	1,819		9.608	0.206	0.033
sc-nasasrb	20	28,154	0.090	TLE	0.301	TLE
	80	834,914		TLE	102.820	0.759
	99	12,742		TLE	0.415	0.079
sc-pkustk11	20	22,612	0.143	TLE	0.645	TLE
	80	2,214,411		TLE	TLE	2.219
	99	24,891		TLE	0.611	0.167
sc-pwtk	20	58,523	0.286	TLE	1.093	TLE
	80	4,483,887		TLE	TLE	3.977
	99	54,907		TLE	1.364	0.421
sc-ldoor	20	274,585	1.679	TLE	4.870	TLE
	80	13,473,317		TLE	563.996	14.944
	99	202,030		TLE	4.845	1.658

Table 2: [RQ1] Median end-to-end seconds; the two specialized methods jointly complete all 24 MSC workloads.

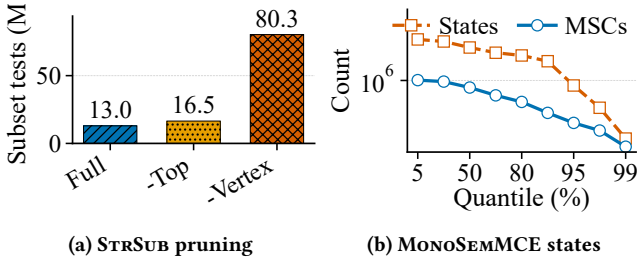


Figure 8: [RQ3] Search bounds and canonical contraction reduce explored states.

Component	Metric / workload	Full	Removed
Top-Edge	tests (M) / nasa q_{99}	12.95	16.50
Vertex bound	tests (M) / nasa q_{99}	12.95	80.28
Local containment	GiB / FB15K q_{80}	0.12	2.26

Table 3: [RQ3] Exact STRSUB ablations reduce subset work or candidate memory without changing outputs.

subset tests from 16.50 to 12.95 million, and the vertex bound reduces them from 80.28 to 12.95 million. On ogbn-arxiv, accepted canonical states contract from 17.08 million at q_5 to 18,011 at q_{99} .

The corresponding runtimes are 0.287 versus 0.324 seconds for Top-Edge, 0.287 versus 0.312 seconds for the vertex bound, and 8.060 versus 11.685 seconds for local containment. Table 4 then holds the feasibility test, similarity cache, and final filter fixed while replacing only duplicate-state organization. Canonical ownership is fastest and uses the least memory on both workloads; every variant returns the same normalized MSC set and size histogram.

Generation	ogbn q_{95}		nasa q_{80}	
	s	GiB	s	GiB
Canonical	0.388	0.39	0.729	0.22
Visited DFS	2.432	0.53	3.069	0.64
Layered	1.635	0.49	3.922	0.81

Table 4: [RQ3] Canonical ownership improves both runtime and memory over visited and layered generation.

6.5 RQ4: MSC Output Space

This experiment answers RQ4 by measuring exact output counts and mean sizes on FB15K-237, ogbn-arxiv, sc-nasasrb, and sc-ldoor over q_{20} , q_{50} , q_{80} , q_{95} , q_{99} . Figure 9 normalizes each value by STRUCTBK on the same graph. The geometric-mean count ratios are 1.04, 7.50, 3.53, 0.61, and 0.11: intermediate thresholds split large structural cliques into many feasible subsets that are maximal by inclusion. Selective thresholds instead contract the result family. Mean size decreases from 1.06 to 0.39 of the structural reference over the same grid.

6.6 RQ5: Group Preservation and External Pattern Families

RQ5 separates a definition comparison from a cross-model comparison. For the definition comparison, the reference cover consists of all MSCs of size at least ten at q_{80} on GitHub-Social and ogbn-arxiv-25K. The response cover contains every non-singleton PAIRSIM-MCE output at the same τ ; retaining small response groups avoids attributing losses to a reporting cutoff. Because both covers overlap, we use Extended B³ Recall [1], whose cluster-completeness property penalizes members of one reference group being separated across response groups. We also report the standard best-match

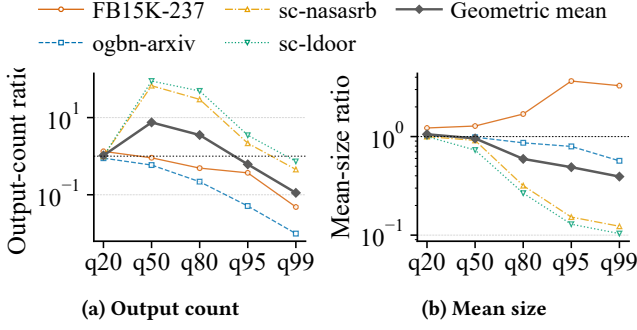


Figure 9: [RQ4] MSCs expand at intermediate thresholds and contract sharply in the selective regime.

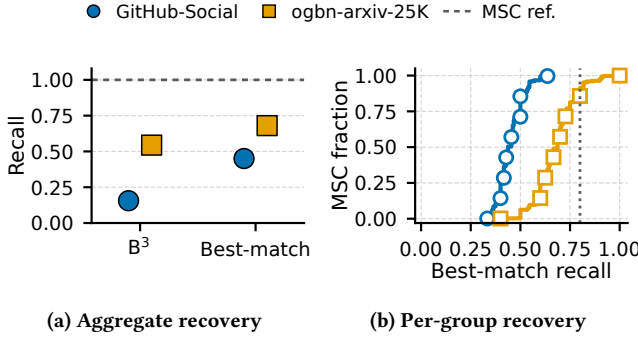


Figure 10: [RQ5] PairSim fragments collectively feasible complete groups. Extended B^3 Recall evaluates the two overlapping covers; best-match recall measures the largest fraction of each MSC retained by one PairSim output. The MSC reference score is one by construction.

community recall [11, 31]

$$R_{\max}(C) = \max_{P \in \mathcal{P}} \frac{|C \cap P|}{|C|},$$

which has the direct interpretation that one PairSim output can retain at most this fraction of MSC C .

Figure 10 shows a pronounced definition effect. On GitHub-Social, PAIRSIM-MCE attains B^3 Recall 0.156 and mean best-match recall 0.450; none of the 605 reference MSCs is retained intact. On ogbn-arxiv-25K, the corresponding scores are 0.544 and 0.680, and only 0.32% of 624 reference MSCs are retained intact. The CDF gives the group-level effect: every GitHub-Social MSC and 84.9% of ogbn-arxiv-25K MSCs lose at least 20% of their members in the best matching PAIRSIM-MCE output. Thus the every-pair constraint fragments groups that remain structurally complete and collectively feasible under the same threshold.

For the external comparison, FastQC at $\gamma = 0.9$ and FaPlex at $k = 2$ are evaluated together with STRUCTBK against the same size-ten MSC reference cover; all result families use the fixed q_{80} vectors and graph. We use the symmetric best-match Average F1 established for overlapping community covers [31], and display its two directional components: *recovery* averages the best-match F1

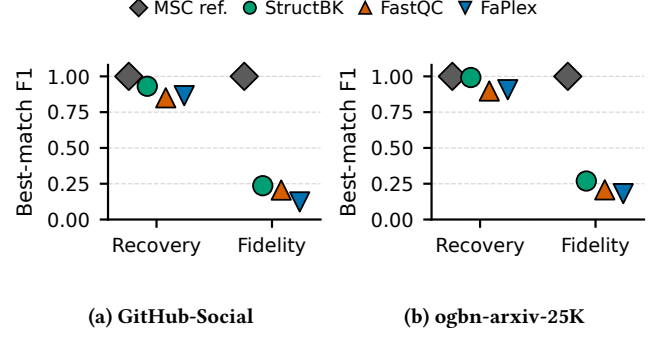


Figure 11: [RQ5] Baselines recover most MSCs but produce many unmatched results.

from each MSC to the baseline cover, whereas *fidelity* averages the best-match F1 from each baseline output to the MSC cover. The mean of these components is symmetric Average F1. This separates recovery of MSCs from proliferation of additional baseline patterns. The comparison measures agreement with the MSC target family, not ground-truth clustering accuracy.

Figure 11 shows that recovery is high for all three baselines, but fidelity is substantially lower. On GitHub-Social, the recovery/fidelity pairs are 0.931/0.236 for STRUCTBK, 0.851/0.206 for FastQC, and 0.865/0.123 for FaPlex. On ogbn-arxiv-25K they are 0.992/0.269, 0.898/0.208, and 0.907/0.181, respectively. High recovery is expected because every MSC is a clique and therefore can be extended to a maximal clique, quasi-clique, or 2-plex; fidelity exposes the extra result families created by that extension. FastQC and FaPlex are not equivalent to STRUCTBK: their pair-weighted edge densities are 0.942–0.952 rather than one, and fewer than 0.05% of their retained outputs are complete cliques. They additionally cover $12.5\times$ – $27.6\times$ more vertices than MSC, with lower pairwise cosine.

Figure 12 provides an instance-level explanation. The largest label-pure MSC at ogbn-arxiv q_{80} contains 18 papers from cs.IT on private information retrieval. Its average similarity is 0.919 at $\tau = 0.889$, although 20 of 153 pairs fall below τ . Collective coherence therefore preserves the complete research group that an every-edge threshold separates.

7 RELATED WORK

Clique Enumeration and Listing. Moon and Moser established the maximal-clique output bound [20]. BK backtracking [3] and Tomita pivoting [26] form the practical foundation, while degeneracy ordering targets sparse graphs [6, 7]. Recent work improves branch selection and termination for MCE [30], develops edge-oriented k -clique listing [29], and summarizes large clique families through comparison operators [18]. These structural techniques motivate the pivoted front end of STRSUB; MSC additionally couples each search state to a non-monotone vector aggregate.

Constrained Cohesive Subgraphs. Dense-subgraph models relax or augment the clique contract in different ways. FastQC enumerates maximal quasi-cliques through co-designed pruning and

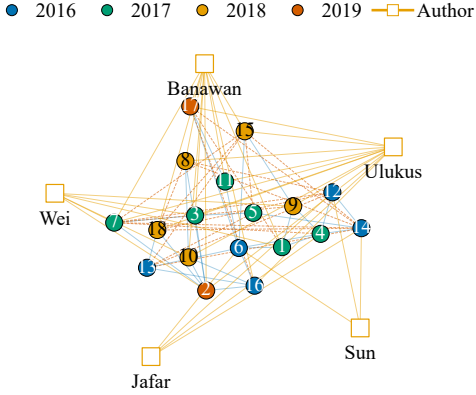


Figure 12: [RQ5] An 18-paper MSC preserves a coherent research group despite 20 sub-threshold pairs.

branching [33], while FaPlex enumerates maximal k -plexes with a worst-case guarantee [35]. Recent exact methods address maximum k -plexes [9], maximal bplexes [4], fair cliques [34], and similar-weight bicliques [32]. MSC retains structural completeness and constrains the group-average vector similarity, so these models provide complementary output families. Our exact efficiency baseline instead preserves the same MSC contract throughout search.

Attributed Community Search. Traditional attributed-graph methods combine topology and attributes under different output contracts. CODICIL augments links with content similarity and partitions the resulting graph [23], whereas ATC returns a query-specific attribute-relevant truss community [13]. Other subgraph methods impose feature-subspace or correlation constraints [10, 21, 24]. Recent systems transfer attributed community search across graphs [8] or use pre-trained graph transformers for unsupervised queries [28]. These methods return a partition or one query-oriented community. MSC instead returns every inclusion-wise maximal, structurally complete group whose collective vector similarity reaches τ .

Vector Attributes. Vector-attributed graphs arise from knowledge-graph embeddings [2, 25], random-walk representations [22], and message-passing models [16]. Approximate vector indexes accelerate unconstrained nearest-neighbor retrieval [14, 19]. MSC evaluates similarities only on graph edges and preserves an inclusive threshold exactly; our implementation therefore precomputes those edge similarities and avoids approximation error inside enumeration.

8 CONCLUSION

We introduced maximal semantic clique (MSC) enumeration under an average pairwise similarity threshold, which identifies structurally complete cliques with guaranteed average pairwise semantic similarity. The problem generalizes classical MCE and is NP-complete. Its defining challenge is the non-monotonicity of the average-similarity constraint over the subset lattice, which invalidates the structural invariants that underpin the efficiency of standard Bron–Kerbosch algorithms.

We used SEMBK as a direct pivot-free baseline and proposed two exact algorithms with distinct operating regions. STRSUB decouples structural and semantic search, restores the Tomita pivot in a τ -agnostic structural phase, and applies admissible edge and vertex-contribution bounds during exact subset enumeration. MONOSEM-MCE is a high- τ specialization: the deterministic deletion rule assigns each feasible clique one parent, so reverse DFS eliminates duplicate insertion paths without visited sets or materialized layers. Both methods use global strict-containment filtering to implement inclusion-wise maximality.

Exact-output tests validate the canonical search and every optional STRSUB component before timing. On ogbn-arxiv, STRSUB is fastest at the intermediate threshold, while MONOSEM-MCE is $4.28\times$ faster at q_{99} ; on FB15K-237, the corresponding high-threshold advantage is $4.48\times$. Across the eight-dataset common grid, STRSUB completes 22 of 24 cells, and canonical search is $2.92\times$ – $6.24\times$ faster on every q_{99} row. The controlled scale series places the crossover at q_{95} across four graph sizes, while canonical edge-root scheduling reaches $5.62\times$ speedup with 32 threads. Mechanism counters connect these results to exact layer bounds in STRSUB and the rapid high-threshold contraction of the canonical tree in MONOSEM-MCE. The fixed five-threshold output study also captures the intermediate pattern-count expansion and high-threshold contraction, while external labels and a real paper group establish the semantic content of the returned MSCs. Future directions include stronger admissible multi-step bounds, finer canonical-subtree scheduling, and dynamic-graph extensions.

REFERENCES

- [1] Enrique Amigó, Julio Gonzalo, Javier Ariles, and Felisa Verdejo. 2009. A Comparison of Extrinsic Clustering Evaluation Metrics Based on Formal Constraints. *Information Retrieval* 12, 4 (2009), 461–486. <https://doi.org/10.1007/s10791-008-9066-8>
- [2] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. 2013. Translating Embeddings for Modeling Multi-Relational Data. In *Advances in Neural Information Processing Systems*. 2787–2795.
- [3] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: Finding All Cliques of an Undirected Graph. *Commun. ACM* (1973), 575–577.
- [4] Qiangqiang Dai, Rong-Hua Li, Donghang Cui, Meihao Liao, Yu-Xuan Qiu, and Guoren Wang. 2024. Efficient Maximal Bplex Enumerations with Improved Worst-Case Time Guarantee. *Proc. ACM Manag. Data* 2, 3 (2024), 135:1–135:26.
- [5] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Software* (2011), 1:1–1:25.
- [6] David Eppstein, Maarten Löffler, and Darren Strash. 2010. Listing All Maximal Cliques in Sparse Graphs in Near-Optimal Time. In *International Symposium on Algorithms and Computation*. Springer, 403–414.
- [7] David Eppstein and Darren Strash. 2013. Listing All Maximal Cliques in Large Sparse Real-World Graphs. *ACM Journal of Experimental Algorithmics* (2013), 3.1:1–3.1:21.
- [8] Shuheng Fang, Kangfei Zhao, Yu Rong, Zhixun Li, and Jeffrey Xu Yu. 2024. Inductive Attributed Community Search: To Learn Communities Across Graphs. *Proc. VLDB Endow.* 17, 10 (2024), 2576–2589.
- [9] Shuohao Gao, Kaiqiang Yu, Shengxin Liu, and Cheng Long. 2024. Maximum k -Plex Search: An Alternated Reduction-and-Bound Method. *Proc. VLDB Endow.* 18, 2 (2024), 363–376.
- [10] Stephan Günnemann, Ines Färber, Brigitte Boden, and Thomas Seidl. 2010. Subspace Clustering Meets Dense Subgraph Mining: A Synthesis of Two Paradigms. In *IEEE International Conference on Data Mining*. IEEE, 845–850.
- [11] Darko Hric, Richard K. Darst, and Santo Fortunato. 2014. Community Detection in Networks: Structural Communities versus Ground Truth. *Physical Review E* 90, 6 (2014), 062805. <https://doi.org/10.1103/PhysRevE.90.062805>
- [12] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *Advances in Neural Information Processing Systems*. 22118–22133.
- [13] Xin Huang and Laks V. S. Lakshmanan. 2017. Attribute-Driven Community Search. *Proceedings of the VLDB Endowment* (2017), 949–960.

- [14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* (2021), 535–547.
- [15] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*. Plenum Press, 85–103.
- [16] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [17] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection.
- [18] Xiaofan Li, Rui Zhou, Lu Chen, and Chengfei Liu. 2025. Clique Comparator: A Fundamental Operator for Finding a Concise Clique Summary. In *ICDE*. 3248–3260.
- [19] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2020), 824–836.
- [20] John W. Moon and Leo Moser. 1965. On Cliques in Graphs. *Israel Journal of Mathematics* (1965), 23–28.
- [21] Flavia Moser, Recep Colak, Arash Rafiey, and Martin Ester. 2009. Mining Cohesive Patterns from Graphs with Feature Vectors. In *SIAM International Conference on Data Mining*. SIAM, 593–604.
- [22] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: Online Learning of Social Representations. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 701–710.
- [23] Yiye Ruan, David Fuhry, and Srinivasan Parthasarathy. 2013. Efficient Community Detection in Large Networks Using Content and Links. In *Proceedings of the 22nd International Conference on World Wide Web*. 1089–1098.
- [24] Arlei Silva, Wagner Meira Jr., and Mohammed J. Zaki. 2012. Mining Attribute-Structure Correlated Patterns in Large Attributed Graphs. *Proceedings of the VLDB Endowment* (2012), 466–477.
- [25] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space. In *International Conference on Learning Representations*.
- [26] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The Worst-Case Time Complexity for Generating All Maximal Cliques and Computational Experiments. *Theoretical Computer Science* (2006), 28–42.
- [27] Kristina Toutanova and Danqi Chen. 2015. Observed versus Latent Features for Knowledge Base and Text Inference. In *Workshop on Continuous Vector Space Models and their Compositionality*. Association for Computational Linguistics, 57–66.
- [28] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Efficient Unsupervised Community Search with Pre-trained Graph Transformer. *Proc. VLDB Endow.* 17, 9 (2024), 2227–2240.
- [29] Kaixin Wang, Kaiqiang Yu, and Cheng Long. 2024. Efficient k -Clique Listing: An Edge-Oriented Branching Strategy. *Proc. ACM Manag. Data* 2, 1 (2024), 7:1–7:26.
- [30] Kaixin Wang, Kaiqiang Yu, and Cheng Long. 2025. Maximal Clique Enumeration with Hybrid Branching and Early Termination. In *ICDE*. 599–612.
- [31] Jaewon Yang and Jure Leskovec. 2013. Overlapping Community Detection at Scale: A Nonnegative Matrix Factorization Approach. In *WSDM*. 587–596. <https://doi.org/10.1145/2433396.2433471>
- [32] Jianye Yang, Lei Xing, Ziyi Ma, Xi Luo, Cuiyun Gao, and Xuemin Lin. 2025. Maximal Similar-Weight Biclique Enumeration for Large Bipartite Graphs. In *ICDE*. 1084–1097.
- [33] Kaiqiang Yu and Cheng Long. 2023. Fast Maximal Quasi-clique Enumeration: A Pruning and Branching Co-Design Approach. *Proc. ACM Manag. Data* 1, 3 (2023), 211:1–211:26.
- [34] Qi Zhang, Rong-Hua Li, Zifan Zheng, Hongchao Qin, Ye Yuan, and Guoren Wang. 2025. Efficient Maximum Fair Clique Search Over Large Networks. In *ICDE*. 1043–1055.
- [35] Yi Zhou, Jingwei Xu, Zhenyu Guo, Mingyu Xiao, and Yan Jin. 2020. Enumerating Maximal k -Plexes with Worst-Case Time Guarantee. In *AAAI*. 2442–2449.